MAESTRÍA EN MINERÍA DE DATOS · UTN FRRo · CICLO 2026

APLICACIONES DE MINERÍA DE DATOS A ECONOMÍA Y FINANZAS

CLASE 4

Feature Engineering, Codificación y Selección de Variables

Construcción de variables con sentido económico · Transformaciones · Codificación categórica · Métodos filter, wrapper y embedded · Diagnóstico de multicolinealidad mediante VIF

UNIDAD 2 · PREPARACIÓN Y EXPLORACIÓN DE DATOS

Jueves 30 de abril de 2026 · 18:00–22:00 hs · Aula Virtual Zoom

Dr. Darío Ezequiel Díaz

APERTURA DE LA CLASE 4

Mapa de navegación de la sesión

1 · Feature engineering

Construcción de variables derivadas con sentido económico-financiero: ratios, variables temporales, agregaciones móviles e interacciones.

2 · Transformación

Normalización min-max, estandarización z-score, transformación logarítmica y familia Box-Cox / Yeo-Johnson.

3 · Codificación

Tratamiento de variables categóricas: one-hot, label, ordinal y target encoding con regularización.

4 · Selección

Tres familias metodológicas: filter, wrapper y embedded. Implementación con Lasso, Random Forest e información mutua.

5 · Multicolinealidad

Diagnóstico mediante el Factor de Inflación de Varianza (VIF). Estrategias de resolución en modelos lineales y de scoring.

6 · Práctica integradora

Notebook guiado en Google Colab sobre el German Credit Dataset con implementación dual en Python y R.

RESULTADO ESPERADO *dominar el flujo completo de preparación de variables hasta dejar el dataset listo para modelar.*

Lo que ya está resuelto: el dataset limpio

01

EDA financiero

Estadísticos descriptivos, distribuciones, correlaciones y diagnósticos visuales sobre el dataset crudo.

02

Valores faltantes

Clasificación según el marco de Rubin (1976): MCAR, MAR y MNAR. Imputación por mediana, KNN y MICE.

03

Outliers

Detección por IQR y z-score modificado (MAD). Tratamiento mediante winsorización, log-transformación o flag binario.

La pregunta que abre la Clase 4

¿Alcanza con un dataset limpio para construir un modelo predictivo competitivo? La respuesta es no. La calidad predictiva depende, en gran medida, de cuán expresivas sean las variables que entreguemos al algoritmo. El feature engineering es donde el conocimiento del dominio económico-financiero deja su huella más profunda sobre el modelo.

Objetivos específicos

- ✓ Construir variables derivadas con relevancia económico-financiera a partir de datos crudos: ratios, indicadores de comportamiento temporal y agregaciones sobre ventanas móviles.
- ✓ Aplicar transformaciones numéricas (min-max, z-score, log, Box-Cox, Yeo-Johnson) y discriminar el método apropiado según la distribución y el algoritmo a entrenar.
- ✓ Dominar las técnicas de codificación de variables categóricas: one-hot encoding, label encoding, ordinal encoding y target encoding regularizado.
- ✓ Implementar los tres paradigmas de selección de variables —filter, wrapper y embedded— en Python y R, evaluando sus trade-offs.
- ✓ Diagnosticar problemas de multicolinealidad mediante el Factor de Inflación de Varianza (VIF) y articular estrategias de resolución coherentes con el modelo objetivo.

BLOQUE 1

Feature engineering con sentido económico

"Coming up with features is difficult, time-consuming, requires expert knowledge. Applied machine learning is basically feature engineering." — Andrew Ng

¿De qué hablamos cuando hablamos de feature engineering?

Definición operativa

El ***feature engineering*** refiere al proceso creativo de construir nuevas variables derivadas a partir de los datos crudos, con el propósito de capturar información predictiva que las variables originales no expresan directamente.

Tres dimensiones lo definen:

- Conocimiento de dominio: traducir hipótesis económicas en variables medibles.
- Operación matemática: razones, diferencias, productos, agregaciones temporales, codificaciones.
- Validación empírica: evaluar el aporte predictivo del feature antes de incorporarlo.

¿POR QUÉ IMPORTA?

La calidad de las features explica más varianza del rendimiento predictivo que la elección del algoritmo.

70 %

del tiempo en proyectos reales se invierte en preparación y construcción de variables (CRISP-DM, Han & Kamber).

+15 %

de mejora típica en AUC al pasar de variables crudas a features económicamente fundadas en scoring crediticio.

Familias de features económico-financieras

RATIOS

Ratios financieros

Cocientes entre magnitudes del balance o de comportamiento. Ejemplos: deuda/patrimonio, cobertura de intereses, loan-to-value, debt-to-income.

credit_amount / duration

TEMPORAL

Comportamiento temporal

Cantidad de atrasos en los últimos 12 meses, variación del saldo mes contra mes, tendencia creciente o decreciente sostenida.

delta_saldo_3m, atrasos_12m

ANTIGÜEDAD

Variables de tenure

Tiempo transcurrido desde un evento focal: días desde la última transacción, meses como cliente, antigüedad laboral.

days_since_last_purchase

ROLLING

Agregaciones móviles

Estadísticos sobre ventanas temporales: media, desvío, máximo y suma del monto transado en los últimos 30, 60 o 90 días.

mean_amount_30d, max_amount_60d

INTERACCIÓN

Variables de interacción

Producto, cociente o concatenación de variables originales para capturar efectos no lineales o segmentaciones implícitas.

ingreso × antigüedad_laboral

BINNING

Discretización por tramos

Conversión de variables continuas en categorías ordinales mediante cortes de negocio (income brackets, edad por décadas).

edad_bin = ['18-25', ...]

Caso aplicado: features para scoring crediticio

VARIABLES CRUDAS

- `credit_amount` (monto solicitado)
- `duration` (plazo en meses)
- `age` (edad del solicitante)
- `employment_years` (antigüedad)
- `checking_status` (saldo cta. cte.)
- `savings_status` (ahorros)

→

FEATURES DERIVADAS

monthly_payment	= credit_amount / duration
debt_to_income	= credit_amount / income
is_young_borrower	= age < 25 → 1
stable_employment	= employment > 4 años
liquidity_score	= checking + savings (ord.)
tenure_x_amount	= antigüedad × monto

→

HIPÓTESIS DE NEGOCIO

- Capacidad de pago mensual
- Endeudamiento relativo
- Mayor riesgo intrínseco
- Estabilidad financiera
- Reserva de liquidez
- Solvencia compuesta

Cada feature derivada es una hipótesis económica codificada en el dataset. La selección posterior dirá cuáles sobreviven al test estadístico.

Implementación dual: Python y R

Python · pandas + numpy

```python
# Ratios financieros
df['monthly_pay']  = df['credit_amount'] / df['duration']
df['debt_to_inc']  = df['credit_amount'] / df['income']

# Variables derivadas
df['is_young']     = (df['age'] < 25).astype(int)
df['stable_emp']   = (df['emp_years'] > 4).astype(int)

# Agregación móvil sobre transacciones
df['amount_30d'] = (df.groupby('id')['amount']
                     .transform(lambda x:
                       x.rolling('30D').mean()))

# Interacción
df['tenure_x_amt'] = df['emp_years'] * df['credit_amount']
```

R · dplyr + lubridate

```r
# Ratios financieros
df <- df %>%
  mutate(monthly_pay = credit_amount / duration,
         debt_to_inc = credit_amount / income)

# Variables derivadas
df <- df %>%
  mutate(is_young   = as.integer(age < 25),
         stable_emp = as.integer(emp_years > 4))

# Agregación móvil con slider
library(slider)
df <- df %>% group_by(id) %>%
  mutate(amount_30d = slide_dbl(amount, mean,
                                .before = 30))
```

La sintaxis cambia; el razonamiento económico permanece. Cada línea materializa una hipótesis sobre el dominio.

BLOQUE 2

Transformación de variables numéricas

Min-Max · Z-score · Logarítmica · Box-Cox · Yeo-Johnson · Quantile transform

Comparativa de transformaciones numéricas

Min-Max

```
x' = (x − x_min) / (x_max − x_min)
```

Reescala al rango [0,1]. Sensible a outliers. Usar cuando el rango es estable y el algoritmo asume escala acotada (redes neuronales, KNN).

Z-score

```
z = (x − μ) / σ
```

Centra en 0 con desvío 1. Apto para algoritmos que suponen normalidad o usan distancias (PCA, regresión logística regularizada, SVM).

Logarítmica

```
x' = log(1 + x)
```

Comprime asimetría positiva, frecuente en variables monetarias (ingreso, monto de crédito, capitalización). Requiere $x \geq 0$.

Box-Cox

```
x' = (x^λ − 1) / λ  (λ ≠ 0)
```

Familia paramétrica que estima λ óptimo por máxima verosimilitud para aproximar la normalidad. Exige $x > 0$.

Yeo-Johnson

```
Generalización de Box-Cox
```

Admite valores negativos y ceros. Adecuada para variables como retornos financieros o variaciones porcentuales.

Quantile

```
F(x) → U(0,1) → Φ⁻¹
```

Mapea cuantiles a una distribución uniforme o normal. Robusta a outliers y monotónica. Usada en modelos no paramétricos.

⚠️ **Regla de oro:** *ajustar la transformación únicamente sobre el conjunto de entrenamiento. Aplicarla luego a validación y test con los parámetros aprendidos. Nunca al revés.*

Decisión metodológica: ¿qué transformación elegir?

ESCENARIO	TRANSFORMACIÓN RECOMENDADA	JUSTIFICACIÓN
Variable monetaria con asimetría positiva (ingreso, monto)	`Logarítmica  log(1+x)`	Comprime cola derecha; preserva interpretación porcentual
Algoritmo basado en distancias (KNN, K-means, SVM)	`Z-score o Min-Max`	Las distancias se distorsionan si las escalas difieren
Regresión lineal con suposición de normalidad	`Box-Cox (x > 0) o Yeo-Johnson`	Aproximan la normalidad y estabilizan la varianza
Algoritmos basados en árboles (RF, XGBoost)	`Ninguna, en principio`	Insensibles a transformaciones monotónicas de las variables
Variable con outliers extremos no removidos	`Quantile transform`	Robusta a outliers; mapea a distribución uniforme o normal
Red neuronal con activaciones acotadas	`Min-Max [0,1] o [-1,1]`	Acelera el entrenamiento; previene saturación de activaciones

BLOQUE 3

Codificación de variables categóricas

One-hot · Label · Ordinal · Target encoding · Binary encoding · Frequency encoding

Mapa de codificadores categóricos

One-Hot Encoding

Crea una columna binaria por categoría. Estándar para modelos lineales. Genera matrices dispersas si la cardinalidad es alta.

✓ Sin orden artificial ✗ Explosión dimensional

Apto para: Reg. logística, SVM, redes neuronales

Label Encoding

Asigna un entero secuencial a cada categoría. Introduce un ordenamiento que puede no existir en la realidad.

✓ Una sola columna ✗ Orden artificial

Apto para: Árboles, Random Forest, XGBoost

Ordinal Encoding

Como label, pero con un orden definido a priori por conocimiento de dominio (educación: primaria<secundaria<universitaria).

✓ Captura jerarquía ✗ Requiere expertise

Apto para: Cualquier modelo si el orden es real

Target Encoding

Reemplaza cada categoría por la media de y condicionada a esa categoría. Captura información predictiva directa.

✓ Alta señal predictiva ✗ Fuga objetivo si no se regulariza

Apto para: Alta cardinalidad (CP, comercio, profesión)

Binary Encoding

Combina label encoding + representación binaria. Reduce dimensionalidad respecto a one-hot manteniendo cierta independencia.

✓ Menos columnas ✗ Difícil de interpretar

Apto para: Variables de cardinalidad media-alta

Frequency Encoding

Reemplaza cada categoría por su frecuencia (relativa o absoluta) en el dataset. Simple y robusta.

✓ Trivial; preserva info ✗ Pierde identidad

Apto para: Etapas exploratorias y modelos rápidos

Target encoding: el riesgo de la fuga del objetivo

Versión regularizada (smoothed target encoding)

$$\mu_cat = (n_cat \cdot \bar{y}_cat + m \cdot \bar{y}_global) / (n_cat + m)$$

`n_cat`: número de observaciones de la categoría · `ȳ_cat`: media local · `ȳ_global`: media global · `m`: peso del prior (típicamente entre 10 y 100)

01 · Validación cruzada interna

Computar la codificación en cada fold de entrenamiento. Nunca con la misma observación que se predice.

02 · Suavizado bayesiano

Mezclar la media de la categoría con la media global. Categorías raras se acercan al prior.

03 · Ruido aditivo

Añadir ruido gaussiano leve durante el entrenamiento atenúa el sobreajuste en categorías de baja frecuencia.

⚠️ *Sin regularización, el target encoding produce sobreajuste severo y AUC engañosamente alto en train que se desploma en validación.*

BLOQUE 4

Selección de variables
filter · wrapper · embedded

"More variables make the curse of dimensionality bite harder." — Hastie, Tibshirani & Friedman, 2009

¿Por qué seleccionar variables?

01

Reducción de dimensionalidad

La maldición de la dimensionalidad: a medida que crecen las variables, los datos se vuelven progresivamente más dispersos en el espacio de features.

02

Mejora de la interpretabilidad

Modelos con menos variables resultan auditables. Crítico en scoring crediticio sujeto a regulación (Basilea, IFRS 9, BCRA).

03

Reducción de tiempos de entrenamiento

Menos columnas implica menor consumo de memoria y CPU/GPU, especialmente relevante en pipelines de producción.

04

Aumento del poder predictivo

Eliminar variables ruidosas o redundantes mejora la generalización al reducir varianza sin incrementar sesgo.

05

Diagnóstico de multicolinealidad

Filtrar variables con VIF elevado evita coeficientes inestables y signos contraintuitivos en modelos lineales.

06

Coherencia con el dominio

Validar que las variables seleccionadas tengan sentido económico-financiero refuerza la confianza institucional en el modelo.

Las tres familias metodológicas

FILTER

Métodos de filtro

Evalúan la relevancia de cada variable de forma independiente del algoritmo de modelado.

MÉTODOS REPRESENTATIVOS

- Pearson · Spearman
- Información mutua
- Chi-cuadrado
- ANOVA F-test
- Wilcoxon / Mann-Whitney

✓ Bajo costo computacional

✗ Ignora interacciones

WRAPPER

Métodos wrapper

Emplean el modelo como caja negra para evaluar subconjuntos de variables.

MÉTODOS REPRESENTATIVOS

- Forward selection
- Backward elimination
- RFE (Recursive Feature Elimination)
- Búsqueda exhaustiva
- Algoritmos genéticos

✓ Alta precisión

✗ Costo computacional alto

EMBEDDED

Métodos embebidos

La selección se integra dentro del proceso de entrenamiento del modelo.

MÉTODOS REPRESENTATIVOS

- Lasso (L1)
- Elastic Net
- Importancia MDI en RF
- Ganancia en XGBoost
- SHAP-based selection

✓ Equilibra costo y precisión

✗ Atado al algoritmo

Métodos filter: criterios estadísticos univariantes

Pearson	$\rho = Cov(X,Y) / (\sigma_X \cdot \sigma_Y)$	X numérica, Y numérica. Mide asociación lineal en [-1, 1].
Spearman	ρ_s sobre rangos	Robusta a outliers; capta monotonía no lineal.
Información mutua	$I(X;Y) = \Sigma\Sigma\ p(x,y) \cdot \log[p(x,y)/(p(x)\cdot p(y))]$	Capta dependencias no lineales arbitrarias entre cualquier par.
Chi-cuadrado	$\chi^2 = \Sigma\ (O_{ij} - E_{ij})^2 / E_{ij}$	X categórica, Y categórica. Test de independencia.
ANOVA F-test	$F = MS_{between} / MS_{within}$	X categórica, Y numérica (o viceversa). Diferencia de medias.

Recomendación: combinar al menos dos criterios filter complementarios y validar con un wrapper o embedded antes de la decisión final.

Métodos wrapper: el modelo como juez

Forward Selection

Comenzar con el conjunto vacío. En cada paso, incorporar la variable que más mejora la métrica de validación. Detener cuando la mejora marginal cae bajo un umbral.

Complejidad: $O(p^2)$

Backward Elimination

Comenzar con todas las variables. En cada paso, eliminar la que menos contribuye al modelo (mayor p-valor o menor importancia). Detener al cumplir un criterio de parada.

Complejidad: $O(p^2)$

Recursive Feature Elim.

Entrenar el modelo, ordenar por importancia, eliminar el k% inferior, y repetir. Implementación estándar en sklearn.feature_selection.RFE y RFECV.

Complejidad: $O(p \cdot k)$

Pseudocódigo · Forward Selection con cross-validation

```
selected = {} ;  remaining = all_features
while remaining and improvement > tol:
    best = argmax_{x ∈ remaining}  CV_score(model, selected ∪ {x})
    if CV_score(selected ∪ {best}) − CV_score(selected) > tol:
        selected ← selected ∪ {best} ;  remaining ← remaining \ {best}
```

Métodos embebidos: selección durante el entrenamiento

Lasso · Regularización L1

Función objetivo:

$$\min_\beta \ (1/2n) \cdot ||y - X\beta||^2 + \lambda \cdot ||\beta||_1$$

La penalización L1 fuerza a que muchos coeficientes **se hagan exactamente cero**. Las variables con coeficiente no nulo conforman el conjunto seleccionado.

El hiperparámetro λ controla la severidad de la penalización: λ → 0 reproduce OLS; λ → ∞ deja todo en cero. Se elige por validación cruzada (LassoCV en sklearn, glmnet en R).

Random Forest · Importancia MDI

Mean Decrease in Impurity (MDI):

$$Imp(X_j) = (1/T) \cdot \Sigma_t \Sigma_{nodos\ splitting\ en\ X_j} \Delta_{impureza}(t)$$

Suma ponderada de la reducción de impureza (Gini o entropía) que cada variable produce a lo largo del bosque.

Alternativa: **Permutation Importance** — más robusta, pero costosa: mide la caída en performance al permutar aleatoriamente cada variable sobre el set de validación.

Buena práctica: usar Lasso para modelos lineales/scoring crediticio; RF importance + permutación para modelos de boosting. Combinar criterios refuerza la robustez de la selección.

BLOQUE 5

Multicolinealidad y Factor de Inflación de Varianza

Cuando dos variables dicen lo mismo, una de ellas miente al modelo lineal.

VIF: el termómetro de la multicolinealidad

Definición y cálculo

El ***Variance Inflation Factor*** cuantifica cuánto se infla la varianza del coeficiente estimado de cada variable a causa de su correlación con las demás predictoras.

$$VIF_j = 1 / (1 - R^2_j)$$

donde R^2_j proviene de regresar X_j sobre todas las demás predictoras del modelo.

Interpretación intuitiva: si VIF = 5, la varianza del coeficiente es 5 veces la que tendría si la variable fuera ortogonal al resto.

Escala de interpretación

VIF = 1

Sin colinealidad

1 < VIF < 5

Aceptable

5 ≤ VIF < 10

Moderada — investigar

VIF ≥ 10

Severa — actuar

⚠️ ***En scoring crediticio regulado, VIF > 5 obliga a justificar técnicamente la inclusión de la variable o a reformular el modelo.***

Estrategias de resolución de multicolinealidad

Eliminación selectiva

Excluir la variable con mayor VIF y recalcular iterativamente. Sencilla, transparente, regulatoriamente aceptable.

Apto para: Modelos lineales y de scoring crediticio.

Combinación lineal

Agregar las variables correlacionadas en una nueva (suma, promedio ponderado, primera componente). Pierde interpretabilidad parcial.

Apto para: Cuando las variables comparten un constructo subyacente.

PCA o Factor Analysis

Proyectar el subespacio colineal sobre componentes ortogonales. Reduce dimensión a costa de perder interpretación directa.

Apto para: Modelos predictivos sin requerimientos regulatorios estrictos.

Regularización (Ridge / Lasso)

Ridge estabiliza coeficientes ante colinealidad sin eliminarlos; Lasso selecciona forzando cero a redundantes.

Apto para: Modelos lineales en producción con muchas variables.

Algoritmos no lineales

Random Forest y XGBoost son insensibles a la colinealidad: distribuyen la importancia entre variables correlacionadas.

Apto para: Si la interpretabilidad estricta no es requisito.

Reformulación de variables

Sustituir variables crudas por ratios o diferencias que rompan la correlación (ej.: deuda/ingreso en vez de deuda e ingreso por separado).

Apto para: Cuando el conocimiento de dominio sugiere una construcción más informativa.

Cálculo del VIF en Python y R

Python · statsmodels

```python
from statsmodels.stats.outliers_influence \
    import variance_inflation_factor

import pandas as pd

# X debe contener solo variables numéricas escaladas
vif = pd.DataFrame()
vif['feature'] = X.columns
vif['VIF'] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]

# Variables a eliminar (umbral 10)
drop = vif.loc[vif['VIF'] >= 10, 'feature']
print(vif.sort_values('VIF', ascending=False))
```

R · car + dplyr

```r
library(car)
library(dplyr)

# Modelo lineal auxiliar
fit <- lm(target ~ ., data = df_num)

# VIF por variable
vif_vals <- vif(fit)
vif_df   <- data.frame(
  feature = names(vif_vals),
  VIF     = as.numeric(vif_vals)
) %>% arrange(desc(VIF))

# Filtrado iterativo (umbral = 10)
drop <- vif_df %>%
  filter(VIF >= 10) %>% pull(feature)
```

Recomendación: aplicar VIF de manera iterativa. Eliminar la variable de mayor VIF, recalcular y repetir hasta que todas queden por debajo del umbral.

PRÁCTICA INTEGRADORA

Notebook guiado en Google Colab

DATASET DE TRABAJO · GERMAN CREDIT (STATLOG) — 1.000 OBS · 20 VAR. · TARGET BINARIO

01

Construcción de features

Generar al menos cinco variables derivadas: monthly_payment, debt_to_amount_ratio, is_young_borrower, stable_employment, liquidity_ordinal.

02

Transformación numérica

Aplicar log1p a credit_amount; estandarizar age, duration y employment con StandardScaler / scale().

03

Codificación categórica

One-hot encoding sobre purpose, housing y job. Target encoding regularizado sobre checking_status.

04

Selección filter + embedded

Información mutua sobre el set codificado y Lasso con LassoCV. Comparar las variables retenidas por ambos métodos.

05

Diagnóstico de VIF

Calcular el VIF sobre el subconjunto numérico. Eliminar iterativamente las variables con VIF ≥ 10 y reportar el ranking final.

Formato de entrega: Notebook ejecutable en Colab (Python o R) con celdas comentadas y conclusión analítica final. Subida a Moodle hasta la Clase 5.

Lo que queda asentado de esta sesión

I

El feature engineering es la pieza que más contribuye al rendimiento predictivo

La calidad de las variables derivadas pesa, en la práctica profesional, más que la sofisticación del algoritmo elegido.

II

Cada transformación tiene un dominio de aplicación

Min-Max, z-score, log y Box-Cox responden a supuestos distintos. La elección depende de la distribución, los outliers y el algoritmo.

III

La codificación categórica condiciona el modelo

One-hot para lineales, label/ordinal para árboles, target encoding regularizado para alta cardinalidad.

IV

La selección combina filter, wrapper y embedded

Convergen tres familias: estadísticas univariantes, exploración por subconjuntos y regularización integrada al entrenamiento.

V

El VIF cierra el diagnóstico antes de modelar

Valores superiores a 5 alertan; superiores a 10 obligan a actuar. Es práctica estándar en validación regulatoria.

CIERRE · DE LA CLASE 4

Tarea y anticipación

ENTREGA · NOTEBOOK DE FEATURE ENGINEERING

Sobre el German Credit Dataset, completar un notebook que incluya:

· Construcción de al menos cinco features derivadas

· Transformación logarítmica y estandarización

· Codificación one-hot y target encoding

· Selección con Lasso e información mutua

· Diagnóstico de VIF con filtrado iterativo

· Conclusión comparativa entre métodos de selección

Formato: Notebook en Colab (Python o R). Entrega: Clase 5.

PRÓXIMA CLASE — JUE 07/05

Unidad 3: Modelos de Clasificación · Regresión Logística

- Regresión logística: fundamentos matemáticos.
- Función sigmoide y máxima verosimilitud.
- Interpretación de coeficientes vía odds ratio.
- Construcción de scorecards crediticias.
- Implementación con sklearn y glm() en R.
- Estándar regulatorio en scoring bancario.

LECTURA SUGERIDA · *Apuntes de Cátedra, Capítulo 3 — Regresión logística y construcción de scorecards. Hosmer & Lemeshow (2013), capítulo introductorio.*

Gracias

Hasta la Clase 5

Dr. Darío Ezequiel Díaz

Director de Metodología y Estadística — IPEC Misiones · Docente UTN FRRo · UCP · UCSF